RAG Pipeline Step 6: Query Construction

1. Definition

Query Construction is an advanced "pre-retrieval" technique that translates a user's natural language query into a *structured, domain-specific query language* that a database can execute.

This is different from **Query Translation** (like Multi-Query), which only changes the natural language. Query Construction fundamentally *changes the structure* of the query from plain text into a formal language like SQL, Cypher, or a metadata filter.

This step is essential when your data has a clear, filterable structure, such as in a SQL database, a graph database, or a vector store with rich metadata.

2. Key Use Cases & LangChain Tools

There are three primary applications for Query Construction:

A. Text-to-SQL (for Structured Databases)

This is the classic example. The goal is to let a user "talk" to a SQL database in plain English.

- **When to use it:** When your data is in a structured database (like SQLite, PostgreSQL, MySQL) and you want to answer analytical questions.
- **User Query:** "What were the total sales for the product 'Banana' in the last 30 days?"
- **How it works (Process):**
  1. **Schema Loading:** An LLM is "grounded" by being given the database schema (table names, column names, and data types).
  2. **Generation:** The LLM receives the user's query and the schema.
  3. **Construction:** It **constructs** a formal SQL query.
  4. **Constructed Query (Output):**

```
SELECT SUM(sales_amount)
FROM sales s
JOIN products p ON s.product_id = p.id
WHERE p.name = 'Banana'
AND s.sale_date >= DATE('now', '-30 days');
```

  5. **Execution:** This SQL query is then executed on the database to get the factual answer.
- **Key LangChain Tools:**
  - `SQLDatabase` **:** A utility wrapper to connect to any SQLAlchemy-compatible database.

- `create_sql_query_chain`: The modern, LCEL-based chain that takes a user query and the DB schema and *only* generates the SQL query string (Source 3.1).
- `create_sql_agent`: A more powerful, complete agent. Unlike the chain, this agent can:
  - Inspect table schemas dynamically.
  - **Self-Correct:** If the generated SQL fails (e.g., syntax error), the agent sees the error message and re-writes the query to fix it (Source 1.4, 1.6).

B. Self-Querying (for Vector DB Metadata Filtering)

This is the use case you correctly identified. It allows you to perform a semantic search *and* a metadata filter at the same time, all from a single natural language query.

- **When to use it:** When your vector store documents are enriched with metadata (like dates, authors, categories, source, etc.) and you want users to be able to filter on it.
- **User Query:** "Find me articles about RAG written by 'Harrison Chase' in 2024."
- **How it works (Process):**
  1. **Schema Definition:** You first define your metadata schema for the LLM using `AttributeInfo` (e.g., describing that "year" is an integer).
  2. **Query Parsing:** The LLM receives the user's query and *parses* it into a `StructuredQuery` object with two distinct parts:
     1. `query`: The semantic part (e.g., "articles about RAG").
     2. `filter`: The structured metadata filter (e.g., `{'author': 'Harrison Chase', 'year': 2024}`).
  3. **Translation:** A `Translator` component converts this generic filter into the specific syntax of your vector store (e.g., Chroma syntax, Pinecone syntax, or Elasticsearch JSON).
  4. **Execution:** The retriever performs a search that must satisfy *both* conditions:
     - Find vectors semantically similar to "articles about RAG"...
     - ...*AND* where the document's metadata `author` field is "Harrison Chase" and `year` is 2024.
- **Key LangChain Tool:**
  - `SelfQueryRetriever`: This is the primary tool. You initialize it by giving it the LLM, the Vector Store, and the `AttributeInfo` list (Source 2.1, 2.2).

C. Text-to-Cypher (for Graph Databases)

This applies if you are using a Knowledge Graph (like Neo4j).

- **When to use it:** When your data is highly interconnected (e.g., fraud detection, social networks) and stored in a graph DB.
- **User Query:** "How is RAG linked to LangChain?"

- **Constructed Query (Cypher):**

```
MATCH (c:Concept {name: "RAG"})-[:IMPLEMENTED_BY]->(f:Framework {name: "LangChain"}
```

- **Key LangChain Tool:** `GraphCypherQAChain` (Source 4.1).

3. Challenges & Best Practices

- **Security (SQL Injection):** When allowing an LLM to write SQL, always restrict the database user permissions to **Read-Only**. Never let the LLM execute `DROP`, `DELETE`, or `UPDATE` commands (Source 3.5).
- **Schema Management:** For massive databases with thousands of tables, you cannot put the entire schema in the prompt. You must use a **retrieval step** first to find only the *relevant* table names before asking the LLM to write the SQL (Source 3.4).
- **Hallucination:** The `SelfQueryRetriever` might try to filter on a metadata field that doesn't exist (e.g., filtering by "mood" when you only have "genre"). providing clear `description` fields in your `AttributeInfo` is critical to prevent this.